

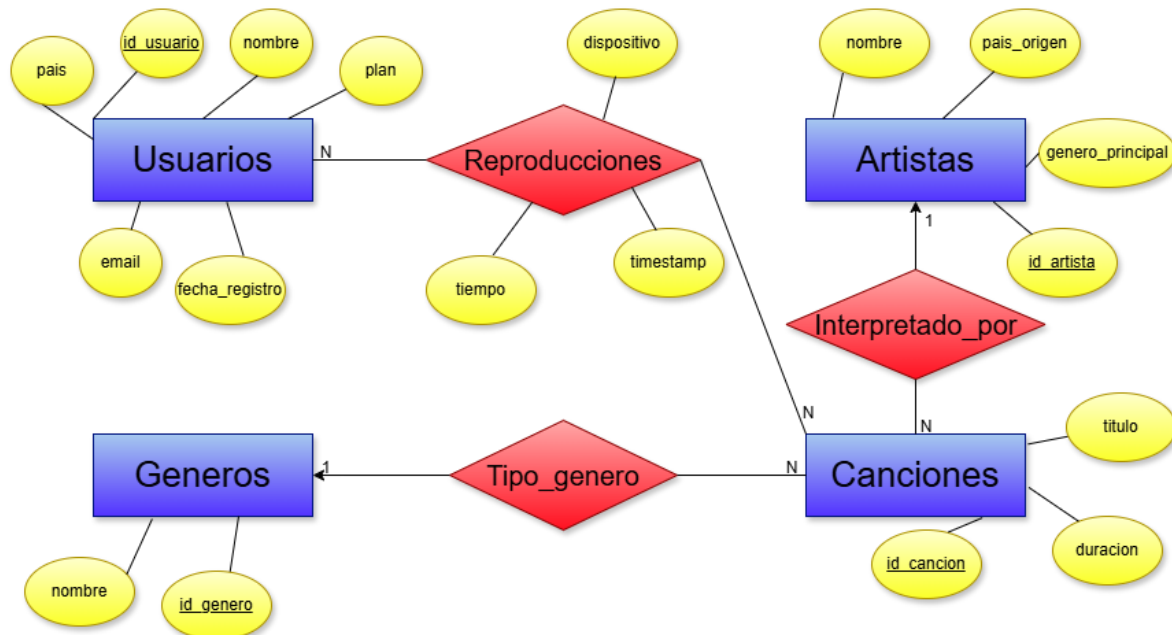
Informe T3 – Parte A–E y **visualizaciones**

María José Cadenas Orta

07 de Julio de 2026

Parte A:

1. Propuesta de diagrama ER para el dominio indicado:



Las entidades del diagrama junto a sus atributos y llaves son:

- Usuarios: id_usuario (PK), nombre, email, pais, fecha_registro, plan.
- Artistas: id_artista (PK), nombre, pais_origen, genero_principal.
- Canciones: id_cancion (PK), duración, titulo.
- Generos: id_genero (PK), nombre.

La relación **Reproducciones** vincula las entidades **Usuarios** y **Canciones**, presentando una cardinalidad N:N. Esta cardinalidad se debe a que un usuario puede reproducir múltiples canciones a lo largo del tiempo y, de igual forma, una misma canción puede ser reproducida por múltiples usuarios. Además, la relación almacena información propia de cada reproducción, como el *timestamp*, el dispositivo utilizado y la cantidad de segundos efectivamente escuchados, atributos que describen el evento de reproducción y no a las entidades involucradas.

Por otro lado, en el diagrama se incorporaron las relaciones **Interpretado_por** y **Tipo_genero**.

La relación **Interpretado_por** vincula las entidades **Artistas** y **Canciones**, con una cardinalidad 1:N, ya que se asumió que un artista puede interpretar múltiples canciones, mientras que cada canción posee un único artista principal. Si bien en plataformas de streaming es posible que una canción sea interpretada por varios artistas (colaboraciones), se optó por esta simplificación debido a que esta decisión

reduce la complejidad del modelo y facilita la posterior transformación al esquema relacional y al esquema estrella.

Por otro lado, **Tipo_genero** relaciona canciones con una cardinalidad de N:1, significando que un mismo género musical puede estar asociado a varias canciones, mientras que cada canción pertenece a un único género. Además, el género no se modeló como un atributo de **Canciones**, sino mediante una relación con la entidad **Géneros**, con el objetivo de mantener un catálogo único de géneros y evitar la duplicación de información. Esto favorece la normalización del esquema y simplifica su mantenimiento.

2. Esquema OLTP en BCNF:

Cada tabla del esquema está en BCNF, ya que en todas ellas la única dependencia funcional no trivial presente es la que va desde la llave primaria hacia el resto de los atributos, y dicha llave primaria constituye por definición una superllave de la relación. No existen dependencias funcionales adicionales entre atributos no-clave en ninguna tabla, por lo que se cumple la condición de BCNF: para toda dependencia funcional no trivial $X \rightarrow Y$, X es superllave.

A continuación, se definen las tablas del sistema. Las Llaves Primarias (PK) se encuentran subrayadas y las Llaves Foráneas (FK) están marcadas con un asterisco (*).

Tablas de entidades:

- Usuarios (id_usuario, nombre, email, pais, fecha_registro, plan)
- Artistas (id_artista, nombre, pais_origen, genero_principal)
- Canciones (id_cancion, duración, titulo, id_artista*, id_genero*)
- Generos: (id_genero, nombre)

Tablas de relaciones y eventos:

- Reproducciones (id_reproduccion, id_usuario*, id_cancion*, timestamp, dispositivo, tiempo)

3. Esquema estrella:

La representación de la tabla de hechos es una reproducción individual: cada fila corresponde a un evento en que el usuario escuchó una canción, con sus métricas y contexto (timestamp, dispositivo, tiempo escuchado) y las llaves foráneas hacia las dimensiones usuario, canción y artista.

En las dimensiones opté por desnormalizar el género directamente como texto plano dentro de Dim_cancion, evitando una tabla de género aparte. En cambio, mantuve Dim_artista como dimensión independiente (en vez de desnormalizarla dentro de Dim_cancion), ya que permite consultas con distinto número de joins sobre el esquema estrella, útil para las clases de consulta de la Parte C.

Tabla de hechos:

- Fact_reproducciones (id_reproduccion, id_usuario*, id_cancion*, id_artista*, timestamp, dispositivo, tiempo)

Tablas de dimensiones:

- Dim_usuario (id_usuario, nombre, email, pais, fecha_registro, plan)
- Dim_artista (id_artista, nombre, pais_origen, genero_principal)
- Dimen_cancion (id_cancion, titulo, duración, genero, id_artista*)

Parte B: Gerenerar datos

Se utilizaron como base los siguientes tamaños por los cuales dependiendo de la escala (1, 5, 10, 50, 1000) estos aumentan generando las 5 instancias. Donde la tabla Reproducciones es la que concentra el mayor crecimiento y se utiliza como referencia principal del tamaño de cada instancia.

Tabla	Tamaño base
Usuarios	1000
Artistas	100
Canciones	1000
Reproducciones	10000

1. Generador sintético: opté por usar varios datos sintéticos como por ejemplo el nombre de usuarios y de artistas que está dado por la combinación de "u+id_usuario"; "a+id_artista" y "c+id_cancion", respectivamente, donde cada id corresponde a un SERIAL generado automáticamente por postgres. Los datos de países fueron generados con la librería externa Faker.

Opté por fijar datos como **GENEROS_FIJOS** = ["rock", "pop", "jazz", "clasica", "reggaeton", "trap", "salsa"] o sea que los únicos géneros disponibles son estos y para escoger cada uno hice la función escoger_genero() declarada en **generador.utils**. Por otro lado, cada generador entrega un dataframe, en el caso de generar_usuarios() y generar_reproducciones() usé numpy para vectorizar la generación de los datos y optimizar un poco la ejecución ya que estas 2 tablas en la última

instancia tienden a crecer demasiado. Además, todos los generadores respetan las relaciones definidas en el esquema de la base de datos. Para esto, las tablas se generan en un orden que permite utilizar los identificadores de las tablas relacionadas al momento de construir las tablas dependientes.

La tabla más relevante que abarca relaciones con más de una tabla es reproducciones y esta se genera a partir de los usuarios y canciones existentes. Para cada reproducción se selecciona un usuario y una canción según los identificadores presentes en las tablas correspondientes. Además, algunos atributos se generan a partir de información de dichas tablas para mantener coherencia entre los datos. Por ejemplo, el valor de tiempo se genera entre 1 y la duración de la canción seleccionada, por lo que una reproducción nunca puede registrar más segundos escuchados que la duración de la canción. Asimismo, el tmsp de una reproducción se genera a partir de la fecha_registro del usuario seleccionado, por lo que la reproducción ocurre después del registro de dicho usuario y nunca antes de este último. De esta manera, las tablas mantienen tanto la integridad referencial como restricciones de coherencia entre sus atributos, evitando generar registros aislados que no tengan correspondencia con las entidades relacionadas. Finalmente, cada generador entrega un dataframe que luego es transformado en un csv con los datos según la instancia y la tabla específica.

2. Transformación OLTP → estrella (ETL): con los datos en los csv primero cargo según la instancia, las tablas a una base de datos con nombre: db_streaming_escala{n} donde n pertenece a [1, 5, 10, 50, 1000], a carga de los datos en PostgreSQL se realiza mediante el comando COPY ... FROM STDIN, utilizando psycopg2 y copy_expert(). Los datos se leen directamente desde los archivos CSV generados previamente y se insertan masivamente en las tablas correspondientes, evitando realizar inserciones fila por fila. Antes de cada carga se ejecuta TRUNCATE ... CASCADE para asegurar que la tabla no contenga datos de ejecuciones anteriores. Luego con los datos ya cargados en Postgres se hace el proceso de ETL. Primeramente en la carpeta etl declare unas consultas sobre el esquema OLTP que va a generar las tablas de dimensiones del OLAP y la tabla de hechos de la misma, entonces se ejecutan una a una las consultas y se exporta ese resultado a un archivo .parquet con el nombre correspondiente según la tabla que corresponde. Sin embargo, el script para generar el archivo .parquet de reproducciones es distinto al del resto de dimensiones porque al ser una tabla que escala tan grande era

muy costoso manejar todos los datos en una sola corrida, por ende decidí usar chunks entonces los .parquet están por parte.

3. Verificación de equivalencia: la conexión con DuckDB se realiza mediante una base de datos temporal en memoria (:memory:). Esto permite crear una instancia independiente de DuckDB para cada ejecución del experimento, sin necesidad de mantener un archivo de base de datos persistente. Las tablas del esquema estrella se cargan directamente desde los archivos Parquet mediante la función read_parquet() de DuckDB. Las tablas de dimensiones se leen desde sus respectivos archivos Parquet, mientras que la tabla de hechos se construye leyendo todos los archivos Parquet almacenados en la carpeta correspondiente. Luego se ejecutan todos los scripts solicitados de equivalencia, donde se imprime en consola el mensaje de si la instancia pasa la prueba o no.

Parte C — Clases de consultas y su traducción:

Las 4 clases de consultas que definí son:

Clase de consulta	Operación	N° de Joins	Selectividad	Agregación
Clase 1: agregación_simple	SELECT	1	baja	1 a 2
Clase 2: alta_selectividad	SELECT	0	alta	0
Clase 3: alta_selectividad_join	SELECT	1	alta	0
Clase 4: joins_agregacion	SELECT	2 a 3	baja	3 a 4

Hipótesis:

En general y viendo nada más como sería la estructura de cada consulta por clase:

1. La clase 2 y 3 ganaría OLTP porque presentará una menor sensibilidad al crecimiento de los datos, ya que la consulta utiliza un filtro altamente selectivo que permite identificar rápidamente una cantidad muy reducida de registros. Al aumentar el volumen total de datos, el costo de la consulta debería mantenerse estable, especialmente porque el atributo que usé para filtrar la primary key que en OLTP es un índice “natural”, pues esta indexado en el B+tree por lo tanto las consultas sobre ese atributo filtrado tienden a ser más eficientes en OLTP que en OLAP ya que este último no posee es índice.
2. La clase 1 y 4 espero que gane OLAP y presente menor sensibilidad al crecimiento de los datos cuando se ejecutan consultas de esta clase. Porque

el esquema estrella está pensado para tener mejor desempeño en consultas analíticas sobre grandes volúmenes de información, facilitando la combinación de la tabla de hechos con sus dimensiones y la ejecución de operaciones como COUNT, SUM y AVG. En cambio, el esquema normalizado de OLTP requiere combinar múltiples tablas para obtener la información necesaria (hacer muchos JOINS que sin índices son muy costosos), aumentando el costo de procesamiento a medida que crece la cantidad de registros. Por lo tanto, se espera que **la curva de OLTP tenga una pendiente mayor que la de OLAP**, evidenciando así que el esquema estrella procesa mejor este tipo de consultas.

Consultas por clase:

Clase	Consulta 1	Consulta 2	Consulta 3
Clase 1: agregación_simple	Artistas con canciones de mayor duración promedio	Reproducción más larga de cada canción	Como se comportan las reproducciones según el plan
Clase 2: alta_selectividad	Datos del usuario de id = 45	Datos del artista con id 100	Datos de la reproducción con id = 100
Clase 3: alta_selectividad_join	Datos de la canción con id = 1000, el nombre del artista y su país de origen	Datos de la reproducción con id = 678 junto al usuario su plan y país	Todas las canciones del artista con id = 50
Clase 4: joins_agregacion	Datos de reproducción según genero considerando reproducciones tiempo total y duración promedio de canciones	Datos de cómo se comportan los usuarios de plan free, junto a la cantidad de usuarios por país, la cantidad de reproducciones, tiempo total y promedio	Datos de como son las reproducciones según el país de origen del artista

Parte D/E - Experimentos de escalabilidad y conclusiones:

Entorno experimental

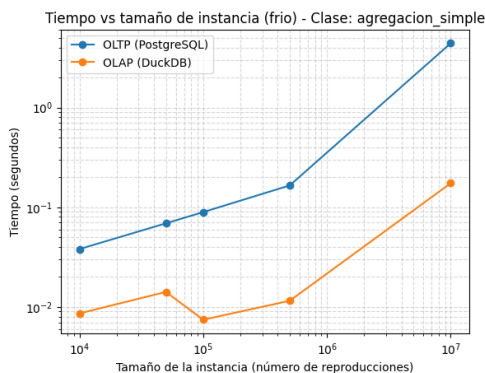
Característica	Especificación
CPU	12th Gen Intel(R) Core(TM) i5-1235U
RAM	7,68 GB
Disco	SSD, 512 GB
PostgreSQL	17.9
DuckDB	1.5.4

Metodología para los experimentos:

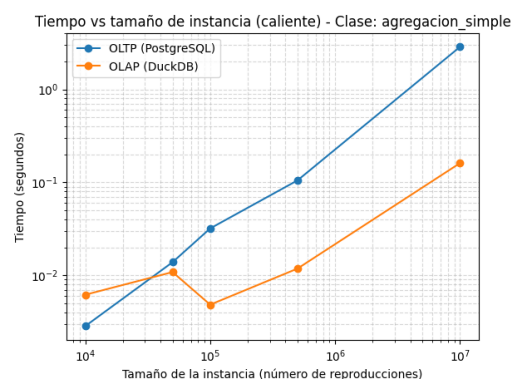
Antes de cada medición se reinició el servicio de PostgreSQL para limpiar la caché asociada al proceso de la base de datos. Luego se ejecuta la consulta correspondiente y se registra su tiempo de ejecución. Para DuckDB se abre una nueva conexión/proceso antes de realizar la medición. Es importante señalar que estas mediciones corresponden a un **estado frío aproximado**, y no a un vaciado completo de todas las cachés del sistema operativo y del almacenamiento. En particular, no se garantiza la eliminación de la caché del sistema de archivos de Windows. Por lo tanto, los resultados deben interpretarse como mediciones en condiciones controladas de caché fría, pero no como un cold cache absoluto.

Resultados consultas de la clase 1 – agregación simple:

En frío: Gana OLAP



En caliente: Gana OLAP

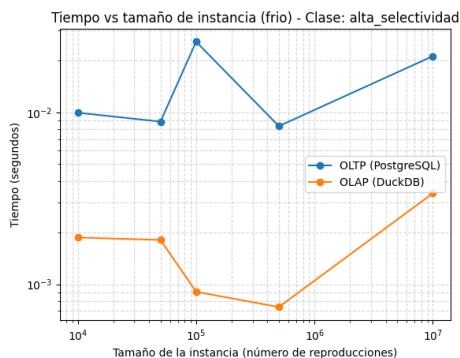


Los resultados coinciden con la hipótesis esta clase de consultas favorece en todas las instancias el esquema OLAP, como fue mencionado anteriormente estas consultas requieren revisar dato a dato para poder hacer las agregaciones (COUNT, MAX, AVG, SUM,...) y sobre demasiados datos DuckDB procesa de forma más efectiva la consulta. En los gráficos se evidencia claramente la diferencia entre las

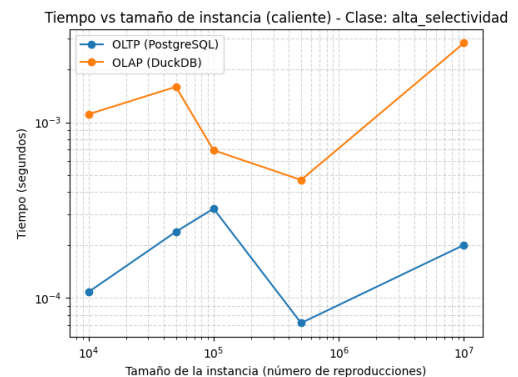
pendientes: a medida que aumenta la cantidad de datos, la pendiente de OLTP es mucho más pronunciada respecto a la pendiente de OLAP y en términos de tiempo la diferencia es interesante notar que en la instancia más pequeña (10^4 filas) ambos motores parten desde tiempos casi idénticos, de aproximadamente 0.003 segundos la diferencia entre ambos es prácticamente nula a esta escala. Sin embargo, a medida que crece el tamaño de la instancia, la diferencia es muy evidente: en la instancia más grande (10^7 filas), PostgreSQL alcanza aproximadamente 4.3 segundos, mientras que DuckDB se mantiene en apenas 0.17 segundos. Esto se traduce en un crecimiento de aproximadamente 1500 veces para PostgreSQL entre la instancia más pequeña y la más grande, mientras que DuckDB crece solo mucho menos en el mismo rango evidenciando que, aunque ambos motores parten en igualdad de condiciones a baja escala, DuckDB escala muchísimo mejor a medida que el volumen de datos aumenta. Las pequeñas fluctuaciones observadas en las instancias intermedias pueden atribuirse a variabilidad experimental y al hecho de que las mediciones no representan un vaciado completo de todas las cachés del sistema. Y esto último también afecta la corrida en frío, pero aún así OLAP sigue siendo más eficiente.

Resultados consultas de la clase 2 – alta selectividad:

En frío: Gana OLAP



En caliente: Gana OLTP

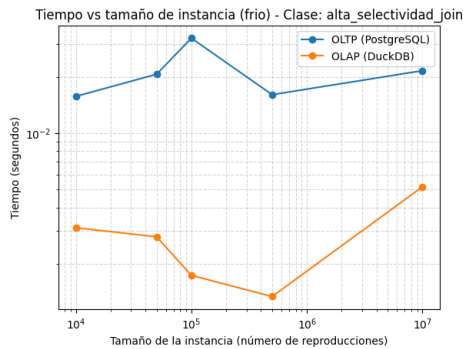


Se observa que ambas curvas mantienen una separación relativamente constante en escala logarítmica a través de las primeras cuatro instancias, con un patrón compartido (un descenso en las escalas intermedias). Es solo en la instancia más grande (10^7) donde se observa una divergencia clara de pendiente: mientras OLTP se mantiene estable, OLAP crece de forma más pronunciada, confirmando que la ventaja estructural del índice (B+tree) de PK en PostgreSQL, se hace más evidente a medida que el volumen de datos crece. Por otro lado, en frío, DuckDB supera a PostgreSQL. Según lo que investigué, esto se explica porque el costo de "enfriar" cada motor no es equivalente. Lo que evidencia una limitación metodológica relevante: la implementación que logré realizar para simular condiciones de frío no

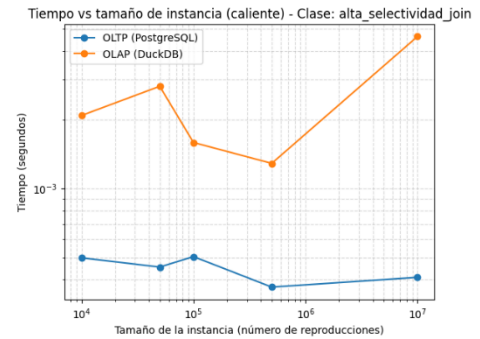
es directamente comparable entre ambos motores, y por tanto el régimen caliente resulta más representativo para evaluar el rendimiento de cada consulta.

Resultados consultas de la clase 3 – alta selectividad join:

En frío: Gana OLAP



En caliente: Gana OLTP

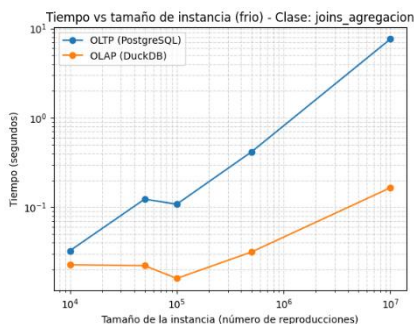


Nuevamente los resultados confirman la hipótesis, PostgreSQL supera abismalmente a DuckDB a diferencia de la clase anterior donde en algunos puntos obtenían tiempos relativamente cercanos. El tiempo de PostgreSQL permanece prácticamente constante en las primeras cuatro instancias, evidenciando nuevamente el comportamiento logarítmico del índice B-tree. DuckDB, en cambio, muestra tiempos consistentemente más altos, reflejando el costo adicional de realizar el join sin un índice tradicional sobre la columna de unión.

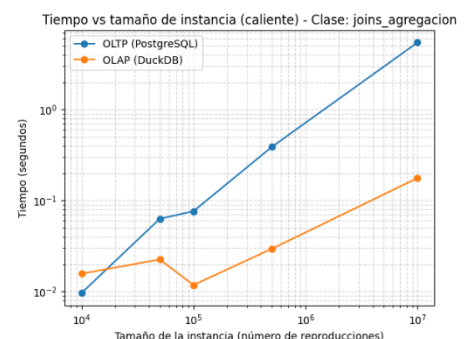
Nuevamente y como fue mencionado antes, en frío gana OLAP, pero por la limitación de la implementación en frío.

Resultados consultas de la clase 4 – joins agregacion:

En frío: Gana OLAP



En caliente: Gana OLTP



En esta última clase, la hipótesis también es confirmada, gana notoriamente OLAP incluso cuando en un principio tienen tiempos muy cercanos. Es a medida que aumentan los datos que se evidencia la diferencia y como la pendiente en ambas curvas aumenta (es creciente), pero para efectos del motor de PostgreSQL este

aumento es muchísimo más pronunciada respecto a OLAP. En la instancia más grande PostgreSQL alcanza casi los 34 segundos, mientras DuckDB apenas 0.39 segundos. Esto se debe a la alta cantidad de JOINS que deben realizarse en estas consultas y evidencia como es que, en OLTP, cada JOIN implica recorrer y cruzar tablas normalizadas, lo que escala considerablemente a medida que los datos aumentan. En el esquema estrella, la desnormalización hace que las consultas como estas requieran menos JOINS y así no crece tanto su ejecución, por ello DuckDB realiza la agregación casi de forma directa.

Conclusión general:

En conclusión, el motor de PostgreSQL es más eficiente cuando se hacen consultas donde se filtre por un atributo sobre el cual existe un índice de **primary key** (B+tree), ya que dicho índice permite localizar la fila buscada en tiempo prácticamente logarítmico sin importar el tamaño de la tabla. Esta ventaja se mantiene si se le agrega un JOIN a la consulta y sea consistente con la selectividad alta. Que se comporte así está muy relacionado con el diseño y con el cómo está pensado PostgreSQL: optimizado para operaciones puntuales y transaccionales sobre registros específicos, respaldadas por estructuras de indexado tradicionales.

El motor DuckDB y el esquema estrella, se ven favorecidos en el escenario opuesto: consultas donde la selectividad es nula o casi nula, o sea, que requiera recorrer casi en su totalidad la tabla de mayor volumen, y sus operaciones principales sean agregaciones con varios JOINS. En este particular, el motor se beneficia de dos factores estructurales: su almacenamiento columnas y la desnormalización del esquema, que reduce la cantidad de JOINS por hacer. Esta combinación explica por qué la brecha de rendimiento a favor de OLAP se amplía dramáticamente en la clase con mayor cantidad de joins y agregación (joins_agregacion), donde alcanzó diferencias de hasta 90 veces en la instancia más grande.

En síntesis, la elección del motor y esquema óptimo depende directamente de la naturaleza de la consulta: para operaciones puntuales sobre registros específicos, identificables mediante llave primaria, PostgreSQL (OLTP) resulta la opción más eficiente; para análisis agregados sobre grandes volúmenes de datos, DuckDB sobre el esquema estrella (OLAP) ofrece un rendimiento sustancialmente superior y, sobre todo, una escalabilidad mucho más favorable a medida que crece el volumen de datos.